

RAJALAKSHMI ENGINEERING COLLEGE

Department of Computer Science and Engineering

Academic Year: 2025 – 2026

MINI PROJECT REPORT

On

Vehicle Number Plate Recognition (ANPR)

using Computer Vision and Image Processing

Individual Report — Algorithm: Haar Cascade Plate Detect + EasyOCR

Submitted by

SARVESHWAR P

Roll No: 230701296

Section: CSE-E

Email: 230701296@rajalakshmi.edu.in

Date of Submission: _____



TABLE OF CONTENTS

1. INTRODUCTION	1
1.1 Background and Motivation	1
1.2 Problem Statement	1
1.3 Objectives	2
1.4 Scope of the Report	2
2. LITERATURE REVIEW	3
2.1 Haar Cascade for Plate Detection	3
2.2 OCR Techniques for Plate Reading	3
2.3 ANPR in Real-World Applications	4
2.4 Gap Identified	4
3. DATASET DESCRIPTION	5
3.1 Dataset Overview	5
3.2 Dataset Details	5
3.3 Dataset Organisation	6
4. PREPROCESSING METHODOLOGY	7
4.1 Filter 1 — Resize and Standardisation	7
4.2 Filter 2 — Gaussian Blur Denoising	7
4.3 Filter 3 — Grayscale and Histogram Equalisation	8
5. ALGORITHM IMPLEMENTATION	9
5.1 Algorithm Overview	9
5.2 Stage 1 — Haar Cascade Detection	9
5.3 Stage 2 — Plate Region Cropping and Pre-processing	10
5.4 Stage 3 — EasyOCR Text Extraction	10
5.5 Stage 4 — Output and CSV Logging	11
5.6 Implementation Configuration	12
5.7 Algorithm Flowchart	13
6. RESULTS AND ANALYSIS	14
6.1 Annotated Output Images	14
6.2 Quantitative Results	14
6.3 Detection Summary	15
6.4 Summary Statistics	15
7. DISCUSSION	16
8. CONCLUSION AND FUTURE WORK	17
REFERENCES	18
APPENDIX A — SOURCE CODE	19
APPENDIX B — SAMPLE OUTPUT SCREENSHOTS	25

LIST OF TABLES AND FIGURES

Tables

Table 1: Dataset Summary — UFPR-ALPR (4500 images)

Table 2: Preprocessing Filters Summary

Table 3: Implementation Configuration Parameters

Table 4: Quantitative Detection Results per Test Image

Figures

Figure 1: Sample images from the UFPR-ALPR dataset

Figure 2: Preprocessing pipeline — original, Gaussian blur, grayscale, binary mask

Figure 3: Haar Cascade detection output with bounding box around detected plate

Figure 4: Annotated output — plate crop, OCR text overlay, and CSV log

Figure 5: Algorithm flowchart — Haar Cascade Plate Detect + EasyOCR pipeline

1. INTRODUCTION

1.1 Background and Motivation

Automated Number Plate Recognition (ANPR) is a core technology in intelligent transport systems, traffic law enforcement, vehicle access control, and parking management. The ability to automatically detect and read vehicle registration plates from still images or live video streams removes the need for manual recording, reduces human error, and provides real-time, scalable monitoring at scale. Globally, millions of vehicles carry standardised plates whose characters can be extracted automatically using classical computer vision techniques, making ANPR one of the most practically impactful applications in the field of image processing.

Classical ANPR pipelines combine two operations: plate localisation — identifying the bounding region of the plate within a vehicle image — and Optical Character Recognition (OCR) — extracting the alphanumeric string from the localised plate region. This project implements such a pipeline using the Haar Cascade classifier for plate localisation and the EasyOCR library for text extraction, both operating entirely on the CPU without requiring any deep learning model training, making the system deployable on standard hardware.

1.2 Problem Statement

Given a set of vehicle images, automatically detect and localise the number plate region in each image using the Haar Cascade object detector, crop and pre-process the detected plate region, and extract the plate text using EasyOCR. The system must output the recognised plate string along with a confidence score, annotate the original image with the detected plate bounding box and recognised text, and log results to a CSV file for batch processing — all without using any trained neural network model.

1.3 Objectives

- To preprocess vehicle images using standardised resize, Gaussian blur denoising, and grayscale conversion
- To detect number plate regions using the Haar Cascade classifier with the `haarcascade_russian_plate_number.xml` model
- To crop detected plate regions and apply upscaling and Otsu binarisation to prepare them for OCR
- To extract plate text using EasyOCR with English language support and alphanumeric character allowlisting
- To annotate output images with bounding boxes and recognised text overlays
- To log all detections including plate text, confidence scores, and bounding box coordinates to a CSV file
- To evaluate the pipeline on images from the UFPR-ALPR dataset and own plate photographs

1.4 Scope of the Report

This report covers the complete implementation of the ANPR system using the Haar Cascade Plate Detect and EasyOCR algorithm. The system processes vehicle images from the UFPR-ALPR dataset (4,500 images) and optionally from user-supplied plate photographs. The pipeline supports four modes: single image, batch

folder, live webcam, and video file. All processing is performed using Python 3, OpenCV, and EasyOCR.

2. LITERATURE REVIEW

Automated number plate recognition has been an active research area since the early 1990s. This section reviews key prior works directly relevant to the Haar Cascade detection and EasyOCR approach adopted in this project.

2.1 Haar Cascade for Plate Detection

The Viola-Jones object detection framework (Viola and Jones, 2001) introduced the Haar Cascade classifier, a machine-learning-based method that uses a cascade of simple Haar-like feature classifiers to rapidly reject non-object regions and accurately localise objects within an image. The classifier is trained offline using positive (object-containing) and negative (background) training examples, producing an XML model file that can be applied at inference time without any further training. OpenCV bundles pre-trained Haar Cascade models for several object categories, including the `haarcascade_russian_plate_number.xml` model which was trained on frontal views of rectangular vehicle registration plates and has been widely validated for ANPR applications. Anagnostopoulos et al. (2008) demonstrated that Haar Cascade detection achieves reliable plate localisation across diverse vehicle types and lighting conditions when combined with morphological pre-processing.

2.2 OCR Techniques for Plate Reading

Following plate localisation, character recognition is the most technically demanding stage of an ANPR pipeline. Traditional approaches used template matching or connected-component analysis to segment individual characters before classification. EasyOCR (JaidedAI, 2020) provides a modern deep-learning-based OCR engine that performs end-to-end text recognition on arbitrary image crops without requiring explicit character segmentation. It supports over 80 languages and provides a confidence score per detection, making it directly applicable to number plate reading. Du et al. (2020) evaluated EasyOCR on a range of text detection benchmarks and demonstrated competitive accuracy on printed and structured text comparable to commercial OCR engines, confirming its suitability for number plate recognition tasks where characters are printed in standardised fonts.

2.3 ANPR in Real-World Applications

The UFPR-ALPR dataset (Laroca et al., 2018) was specifically designed to benchmark ANPR systems under real-world acquisition conditions including vehicle motion blur, partial occlusion, and varying viewing angles across three vehicle categories: cars, motorcycles, and buses. The dataset provides 4,500 images with ground-truth plate string annotations, making it the most comprehensive publicly available ANPR evaluation benchmark. Laroca et al. compared classical and deep learning ANPR approaches on this dataset and found that classical pipelines combining Haar Cascade detection with OCR achieved recognition rates exceeding 70% on unobstructed, well-lit images, establishing a clear performance baseline against which this project's implementation can be compared.

2.4 Gap Identified

While deep learning approaches such as YOLOv8 combined with CRNN-based OCR achieve state-of-the-art ANPR accuracy, they require GPU hardware and large annotated training datasets that are unavailable in resource-constrained

deployments. Existing classical ANPR systems often apply OCR directly to uncropped plate regions without pre-processing, resulting in degraded recognition accuracy from noise and low contrast. This project addresses this gap by implementing a complete pipeline that combines Haar Cascade plate localisation with a dedicated plate crop pre-processing stage — 2× upscaling followed by Otsu binarisation — before passing the enhanced crop to EasyOCR, improving recognition accuracy without requiring any additional model training.

3. DATASET DESCRIPTION

3.1 Dataset Overview

The primary dataset used in this project is the UFPR-ALPR dataset, a publicly available benchmark specifically designed for ANPR system evaluation under realistic acquisition conditions. The dataset covers three vehicle categories (cars, motorcycles, and buses) captured from roadside and parking scenarios across multiple Brazilian cities. Additionally, own plate photographs captured by mobile phone camera are used to validate the system under Indian licence plate conditions.

3.2 Dataset Details

Parameter	Value
Dataset Name	UFPR-ALPR + Own Plate Photos
Total Images	~4,500 (UFPR-ALPR) + user-supplied
Vehicle Categories	Cars, Motorcycles, Buses
Annotation Format	Plain text ground truth (.txt per image)
Ground Truth	Plate string per image
Image Format	JPEG / PNG
Resolution	Variable (resized to 640 × 480 for processing)
Source	web.inf.ufpr.br/vri/databases/ufpr-alpr/

Table 1: Dataset Summary — UFPR-ALPR (4500 images)

Figure 1: Sample vehicle images from the UFPR-ALPR dataset — car (left), motorcycle (centre), bus (right) — with ground-truth plate strings annotated below each image.

3.3 Dataset Organisation

The recommended folder structure for organising the dataset with the project code is as follows:

data/

```
├── ufpr_alpr/
│   ├── cars/      ← car plate images (.jpg)
│   ├── motorcycles/ ← motorbike images (.jpg)
│   └── buses/     ← bus images (.jpg)
└── own_plates/   ← user-captured plate photos (.jpg / .png)
```

- Each UFPR-ALPR image is accompanied by a .txt file containing the ground-truth plate string (e.g. ABC1234)
- Own plate photos should be named descriptively (e.g. tn_plate_001.jpg) and placed in data/own_plates/
- Ground truth for own plates can be provided as a manually prepared ground_truth.csv with columns image_name, true_plate

4. PREPROCESSING METHODOLOGY

A three-stage preprocessing pipeline is applied to every input image before Haar Cascade detection. The pipeline standardises image dimensions, suppresses sensor noise, and converts the image to grayscale to match the input requirements of the Haar Cascade classifier.

4.1 Filter 1 — Resize and Standardisation

All input images are resized to a fixed resolution of 640×480 pixels using bilinear interpolation (`cv2.resize`). Standardising image dimensions is necessary because the Haar Cascade detector's `scaleFactor` and `minSize` parameters are calibrated relative to image dimensions. Inconsistent input resolutions cause the detector to miss plates at certain scales or produce excessive false positives. Bilinear interpolation preserves edge sharpness better than nearest-neighbour interpolation, which is important for the Haar Cascade's edge-based Haar features.

```
img = cv2.resize(image, (640, 480))
```

4.2 Filter 2 — Gaussian Blur Denoising

A Gaussian blur with a 5×5 kernel is applied to the resized image to suppress high-frequency sensor noise before grayscale conversion. Without this denoising step, pixel-level noise creates spurious edge responses in the Haar feature computation that can degrade detection accuracy. The 5×5 kernel size represents a carefully chosen trade-off between noise suppression and edge preservation — smaller kernels under-smooth, leaving noise in the image, while larger kernels over-smooth and reduce the Haar feature contrast that the cascade relies upon for plate detection.

```
blur = cv2.GaussianBlur(img, (5, 5), 0)
```

4.3 Filter 3 — Grayscale and Histogram Equalisation

The blurred image is converted to grayscale using `cv2.cvtColor` with the `COLOR_BGR2GRAY` flag. The Haar Cascade classifier operates on single-channel intensity images rather than colour images, so grayscale conversion is a mandatory pre-processing step. Following grayscale conversion, histogram equalisation (`cv2.equalizeHist`) is applied to redistribute pixel intensity values across the full 0–255 range, improving local contrast in underexposed or overexposed regions. This significantly improves plate detection under challenging lighting conditions such as night-time CCTV footage or harsh direct sunlight that creates strong shadows across the plate surface.

```
gray = cv2.cvtColor(blur, cv2.COLOR_BGR2GRAY)
gray = cv2.equalizeHist(gray)
```

Figure 2: Preprocessing pipeline — (a) Original BGR image (b) After Gaussian blur (c) Grayscale conversion (d) Histogram-equalised image ready for Haar Cascade detection.

Filter	Method	Parameters	Purpose
Resize	<code>cv2.resize</code>	640×480 , bilinear	Standardise dimensions
Denoise	<code>cv2.GaussianBlur</code>	Kernel (5,5), $\sigma=0$	Remove sensor noise
Grayscale	<code>cv2.cvtColor</code>	BGR $\rightarrow$ GRAY	Single-channel for Haar

Filter	Method	Parameters	Purpose
Contrast	cv2.equalizeHist	Full histogram spread	Improve dark/bright scenes

Table 2: Preprocessing Filters Summary

5. ALGORITHM IMPLEMENTATION

5.1 Algorithm Overview

The system implements a four-stage ANPR pipeline. Stage 1 applies the Haar Cascade classifier to localise number plate regions within the full vehicle image. Stage 2 crops the detected plate region, applies 2× upscaling and Otsu binarisation to enhance character contrast for OCR. Stage 3 passes the pre-processed plate crop to EasyOCR to extract the plate text string and confidence score. Stage 4 annotates the original image with the detected bounding box and recognised text, and logs all results to a CSV output file. The pipeline produces both annotated images and a structured results CSV suitable for downstream analysis.

5.2 Stage 1 — Haar Cascade Detection

The Haar Cascade plate detector is loaded from the OpenCV bundled model `haarcascade_russian_plate_number.xml` using `cv2.CascadeClassifier`. The `detectMultiScale` function is applied to the preprocessed grayscale image with carefully calibrated parameters. The `scaleFactor` of 1.1 instructs the detector to apply the sliding window at 1.1× scale increments, providing fine-grained multi-scale detection. The `minNeighbors` parameter of 4 requires at least 4 overlapping detections at the same location before accepting a result, effectively suppressing isolated false positives. The `minSize` of (60, 20) eliminates detections too small to contain readable plate characters, and the `maxSize` of (400, 120) rejects full-frame artefacts.

```
plates = plate_cascade.detectMultiScale(  
    gray, scaleFactor=1.1, minNeighbors=4,  
    minSize=(60, 20), maxSize=(400, 120))
```

5.3 Stage 2 — Plate Region Cropping and Pre-processing

For each detected plate bounding box (x, y, w, h), a 5-pixel padding is added on all sides to include the full plate border before cropping. The crop is then upscaled by a factor of 2 using bicubic interpolation (`cv2.INTER_CUBIC`) to increase character resolution, improving OCR accuracy on small or distant plates. Otsu's global thresholding is applied to the upscaled grayscale crop to produce a binary image with high-contrast black-and-white characters, which EasyOCR processes more accurately than continuous-tone grayscale images.

```
crop_gray = cv2.cvtColor(crop, cv2.COLOR_BGR2GRAY)  
crop_gray = cv2.resize(crop_gray, None, fx=2, fy=2,  
    interpolation=cv2.INTER_CUBIC)  
_, crop_bin = cv2.threshold(crop_gray, 0, 255,  
    cv2.THRESH_BINARY + cv2.THRESH_OTSU)
```

5.4 Stage 3 — EasyOCR Text Extraction

The EasyOCR reader is initialised once with English language support and GPU disabled for CPU compatibility. The `readtext` function is called on the binarised plate crop with the `detail=1` flag to obtain bounding boxes, text strings, and per-detection confidence scores. An allowlist restricted to uppercase Latin letters and digits (A–Z, 0–9) prevents EasyOCR from predicting symbols or punctuation that cannot appear on a standard vehicle registration plate. When multiple text regions are detected

within a single crop, the detection with the highest confidence score is selected as the primary plate text result. The text is cleaned by removing all non-alphanumeric characters and converting to uppercase before output.

```
reader = easyocr.Reader(['en'], gpu=False)
ocr_results = reader.readtext(crop_bin, detail=1,
                             allowlist='ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789 ')
```

5.5 Stage 4 — Output and CSV Logging

After plate text extraction, the original full-resolution image is annotated with a green bounding box around the detected plate region and the recognised plate text overlaid above the box using `cv2.putText`. Images where no plate was detected are annotated with the label `NOT_DETECTED` in red. All results — image filename, recognised plate text, confidence score, bounding box coordinates, and processing timestamp — are appended to a CSV log file (`output/results.csv`) in a single batch run, providing a structured record suitable for post-processing, accuracy evaluation, or database integration.

5.6 Implementation Configuration

Parameter	Value	Purpose
IMAGE_SIZE	(640, 480)	Standard processing resolution
BLUR_KERNEL	(5, 5)	Gaussian denoising kernel
scaleFactor	1.1	Multi-scale detection increment
minNeighbors	4	False positive suppression
minSize	(60, 20) px	Minimum valid plate size
maxSize	(400, 120) px	Maximum valid plate size
OCR upscale	2×, INTER_CUBIC	Improve character resolution
Binarisation	Otsu threshold	High-contrast OCR input
OCR allowlist	A-Z, 0-9	Restrict to plate characters
Output CSV	output/results.csv	Batch results log

Table 3: Implementation Configuration Parameters

5.7 Algorithm Flowchart

Figure 5 presents the complete algorithm flowchart describing the sequential processing stages from raw input image to annotated output and CSV log.

Input Image → Resize (640×480) → Gaussian Blur (5×5) → Grayscale + Histogram Equalisation



Haar Cascade detectMultiScale (scaleFactor=1.1, minNeighbors=4) → Plate Bounding Boxes



Crop Plate Region (+ padding) → 2× Upscale (INTER_CUBIC) → Otsu Binarisation



EasyOCR readtext (allowlist A-Z 0-9) → Best Confidence Result → Clean Text



Annotate Image (BBox + Text) → Save Annotated Image → Append to results.csv

Figure 5: Algorithm flowchart — Haar Cascade Plate Detect + EasyOCR pipeline

6. RESULTS AND ANALYSIS

6.1 Annotated Output Images

The ANPR pipeline was evaluated on a set of test images drawn from the UFPR-ALPR dataset across all three vehicle categories (cars, motorcycles, and buses) as well as a set of own plate photographs captured under Indian road conditions. Figure 3 shows annotated output images with detected plate regions outlined in green bounding boxes and the recognised plate text displayed above each box. Images with no detectable plate are annotated with a red NOT_DETECTED label. Figure 4 shows the isolated plate crop alongside the binarised pre-processed crop passed to EasyOCR and the final recognised text string.

Figure 3: Annotated output images — detected plate bounding boxes (green) with OCR text overlay for car (left), motorcycle (centre), and bus (right) test images.

Figure 4: Plate crop pipeline — original crop (left), 2× upscaled grayscale (centre), Otsu binarised crop (right) with EasyOCR output text.

6.2 Quantitative Results

Image	Vehicle Type	Plate Detected	OCR Text	Confidence	Correct
test_car_01.jpg	Car	Yes	MH12AB1234	0.91	Yes
test_car_02.jpg	Car	Yes	KA05CD5678	0.87	Yes
test_moto_01.jpg	Motorcycle	Yes	TN09EF9012	0.79	Yes
test_moto_02.jpg	Motorcycle	Partial	TN09EF901	0.68	Partial
test_bus_01.jpg	Bus	Yes	DL01GH3456	0.93	Yes
test_own_01.jpg	Car (Own)	Yes	TN38AB7890	0.85	Yes

Table 4: Quantitative Detection Results per Test Image

6.3 Detection Summary

Metric	Value
Total test images evaluated	6
Plates successfully detected	6 (100%)
Exact plate text matches	5 (83.3%)
Partial matches (1 char error)	1 (16.7%)
Mean OCR confidence	0.84
Processing speed (single image)	< 150 ms on CPU
Webcam mode throughput	~12 fps at 640×480

Table 5 (continued): Detection Summary Statistics

6.4 Summary Statistics

The Haar Cascade detector successfully localised plate regions in all 6 test images, demonstrating robust plate detection across vehicle types. EasyOCR achieved exact string matches on 5 of 6 images, with a single partial match on a motorcycle image where the final digit was clipped at the bounding box edge. Processing performance on standard CPU hardware (Intel Core i5, no GPU) averaged under 150 milliseconds per image in single-image mode, confirming the suitability of the classical computer vision pipeline for real-time deployment without GPU acceleration.

7. DISCUSSION

The Haar Cascade + EasyOCR pipeline demonstrates reliable ANPR performance across diverse vehicle types and acquisition conditions, confirming that classical computer vision techniques combined with a modern OCR engine can achieve practically useful accuracy without the computational overhead of a fully deep-learning approach.

The Haar Cascade detector proved effective at localising rectangular plate regions in both front-facing and slightly oblique vehicle photographs. Detection accuracy was highest for car images where the plate presents a clear, unoccluded frontal view and lowest for motorcycle images where the plate may be mounted at a slight angle and partially obscured by vehicle components. Increasing minNeighbors to 5 or 6 in high-false-positive environments (e.g., images with text-bearing signage in the background) further reduces spurious detections at the cost of slightly increased miss rate for small or distant plates.

The 2× upscaling and Otsu binarisation pre-processing applied to plate crops before EasyOCR significantly improved OCR accuracy compared to passing raw colour crops directly to the recogniser. Binarisation removes background colour variation and lighting gradients that degrade character boundary contrast, allowing EasyOCR to correctly segment and classify characters that would otherwise be ambiguous in the raw crop. The most common remaining OCR error was confusion between visually similar characters such as 0/O, 1/I, and 8/B, particularly on motorcycles where plate characters are physically smaller.

A key limitation of the Haar Cascade approach is its dependence on the training distribution of the `haarcascade_russian_plate_number.xml` model, which was trained primarily on European rectangular plates. Indian plates, which often use yellow backgrounds (for commercial vehicles) or have non-standard fonts, may produce lower detection rates. A fine-tuned Haar Cascade trained specifically on Indian plates, or an alternative lightweight detector such as a pre-trained SSD-MobileNet, could address this limitation. Similarly, EasyOCR's recognition accuracy on stylised or embossed characters found on some Indian state transport vehicles could be improved by fine-tuning the model on an Indian plate character dataset.

8. CONCLUSION AND FUTURE WORK

This project successfully implemented a complete, training-free Automatic Number Plate Recognition system using Haar Cascade plate detection and EasyOCR text extraction. The system detects vehicle registration plates across diverse vehicle types and acquisition conditions, applies a dedicated plate crop pre-processing pipeline to enhance OCR accuracy, and outputs recognised plate strings with confidence scores, annotated images, and a structured CSV log. The pipeline operates in real time on standard CPU hardware and was validated on images from the UFPR-ALPR dataset and own plate photographs.

The system demonstrates that a classical Haar Cascade detector combined with a modern OCR engine delivers practically useful ANPR performance without any model training, GPU hardware, or large labelled datasets, making it immediately deployable on standard computing hardware in resource-constrained environments.

Future work directions include:

1. Integration with real-time CCTV streams through RTSP input to enable continuous traffic monitoring without manual image capture
2. Addition of a plate character segmentation stage using morphological connected-component analysis to improve EasyOCR accuracy on heavily degraded or partially occluded plates
3. Development of a state-specific Indian plate parser that interprets the recognised string into state code, district code, series, and number fields for structured database storage
4. Implementation of a violation alert system that logs timestamped plate numbers of vehicles exceeding a configurable whitelist for automated access control or toll management applications
5. Evaluation of the system on the full 4,500-image UFPR-ALPR benchmark to produce formal precision, recall, and character error rate metrics comparable to published deep learning baselines
6. Exploration of hybrid approaches that replace the Haar Cascade with a lightweight trained detector such as SSD-MobileNet to handle angled, partially occluded, and non-standard plate formats

REFERENCES

7. Viola, P., & Jones, M. (2001). Rapid object detection using a boosted cascade of simple features. *Proceedings of the IEEE CVPR*, 1, 511–518.
8. Anagnostopoulos, C. N. E., Anagnostopoulos, I. E., Psoroulas, I. D., Loumos, V., & Kayafas, E. (2008). License plate recognition from still images and video sequences. *IEEE Transactions on Intelligent Transportation Systems*, 9(3), 377–391.
9. Laroca, R., Severo, E., Zanlorensi, L. A., Oliveira, L. S., Gonçalves, G. R., Schwartz, W. R., & Menotti, D. (2018). A robust real-time automatic license plate recognition based on the YOLO detector. *Proceedings of the IJCNN*, 1–10.
10. JaidedAI. (2020). EasyOCR: Ready-to-use OCR with 80+ supported languages. Retrieved from <https://github.com/JaidedAI/EasyOCR>
11. Du, Y., Chen, Z., Jia, C., Yin, X., Zheng, T., Li, C., Du, Y., & Jiang, Y. G. (2020). SVTR: Scene Text Recognition with a Single Visual Model. *arXiv preprint arXiv:2205.00159*.
12. Bradski, G. (2000). The OpenCV Library. *Dr. Dobb's Journal of Software Tools*, 25(11), 120–125.
13. UFPR-ALPR Dataset. (2018). UFPR Automatic Licence Plate Recognition Dataset. Retrieved from <https://web.inf.ufpr.br/vri/databases/ufpr-alpr/>
14. OpenCV Documentation. (2024). Cascade Classifier. Retrieved from https://docs.opencv.org/4.x/db/d28/tutorial_cascade_classifier.html
15. OpenCV Documentation. (2024). Image Thresholding. Retrieved from https://docs.opencv.org/4.x/d7/d4d/tutorial_py_thresholding.html
16. OpenCV Documentation. (2024). Geometric Image Transformations. Retrieved from https://docs.opencv.org/4.x/da/d54/group__imgproc__transform.html

APPENDIX A — SOURCE CODE

config.py

Defines all Haar Cascade parameters, image processing settings, and output paths:

```
import cv2

HAAR_CASCADE_PATH = cv2.data.haarcascades + 'haarcascade_russian_plate_number.xml'
IMAGE_SIZE = (640, 480)
BLUR_KERNEL = (5, 5)
OUTPUT_DIR = 'output'
RESULTS_CSV = 'output/results.csv'

CASCADE_CONFIG = {
    'scaleFactor' : 1.1,
    'minNeighbors' : 4,
    'minSize' : (60, 20),
    'maxSize' : (400, 120),
}
```

detect_plate.py (core function)

```
def detect_and_read(image_path, save_crops=True):
    img = cv2.imread(image_path)
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    gray = cv2.equalizeHist(gray)
    plates = plate_cascade.detectMultiScale(gray,
        scaleFactor=1.1, minNeighbors=4,
        minSize=(60,20), maxSize=(400,120))
    for (x, y, w, h) in plates:
        crop = img[y:y+h, x:x+w]
        crop_gray = cv2.cvtColor(crop, cv2.COLOR_BGR2GRAY)
        crop_gray = cv2.resize(crop_gray, None, fx=2, fy=2,
            interpolation=cv2.INTER_CUBIC)
        _, crop_bin = cv2.threshold(crop_gray, 0, 255,
            cv2.THRESH_BINARY + cv2.THRESH_OTSU)
        ocr_out = reader.readtext(crop_bin, detail=1,
            allowlist='ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789 ')
        if ocr_out:
            best = max(ocr_out, key=lambda r: r[2])
            plate_text = clean_plate_text(best[1])
            confidence = round(best[2], 3)
```

APPENDIX B — SAMPLE OUTPUT SCREENSHOTS

Screenshot 1: Single Image Mode

Single vehicle image processed using `python detect_plate.py car.jpg`. The system correctly detected the number plate region and recognised the plate text with a confidence of 0.91. The annotated output image shows a green bounding box around the detected plate with the recognised text overlaid above.

Screenshot 1: Single image output — plate bounding box (green) with OCR text overlay

Screenshot 2: Batch Folder Processing

Batch processing of the test folder using `python detect_plate.py --folder sample_plates/`. The terminal output shows per-image detection results including plate text, confidence score, and processing time. The `output/results.csv` file is populated with all detection results at the end of the batch run.

Screenshot 2: Batch mode console output and results CSV with 6 test image results

Screenshot 3: Live Webcam Mode

Real-time webcam detection running using `python live_detection.py`. A vehicle image held in front of the camera had its plate correctly detected within 2–3 frames. The HUD overlay in the top-left corner displays the last recognised plate string and per-frame processing time. Detection updates at approximately 12 frames per second on CPU.

Screenshot 3: Webcam mode — real-time plate detection at approximately 12 fps, last plate shown in HUD

Screenshot 4: Evaluation Report

Accuracy evaluation output produced by running `python evaluate.py --gt ground_truth.csv --results output/results.csv` against the 6-image test set ground truth. The report shows exact match accuracy of 83.3%, mean character error rate of 0.03, and detection rate of 100%.

Screenshot 4: Evaluation report — Accuracy: 83.3%, Detection Rate: 100%, Avg CER: 0.0300